

THỰC HÀNH

GIẢI THUẬT XỬ LÝ SONG SONG VÀ PHÂN BỐ

GVHD:

LÊ MINH KHÁNH HỘI
LÊ PHẠM HOÀNG TRUNG

Ngày 31 tháng 3 năm 2026

1 Mục tiêu

- Hiểu và vận dụng các khái niệm cơ bản về lập trình song song để giải quyết một bài toán tính toán có độ phức tạp cao.
- Nắm vững quy trình thiết kế, hiện thực hóa và đánh giá hiệu năng của một ứng dụng song song

2 Yêu cầu bài toán

2.1 Tính tổng mảng số nguyên

Cho mảng A gồm N số thực ($1 \leq N \leq 10^6$, $|A_i| \leq 10^9$). Viết hàm `MAIN(input_file_path)` đọc dữ liệu từ tệp, chỉ cộng những phần tử đồng thời thỏa mãn:

- Là số nguyên về mặt giá trị toán học (tức là x là số thực nhưng $x = \text{int}(x)$, ví dụ 12.0 cũng được tính là số nguyên).
- Lớn hơn 0.
- Chia hết cho 3.

Các giá trị còn lại bị bỏ qua. Output là một số nguyên duy nhất biểu diễn tổng thỏa điều kiện; cần lưu ý kết quả có thể vượt 32-bit nên phải dùng kiểu số lớn hơn.

Input: Dòng đầu là N . Các dòng sau (có thể gộp một dòng) liệt kê N giá trị thực. Ví dụ:

```
6
3 4.5 -6 12.0 7 18
```

Output: Một số nguyên, ví dụ mẫu trên trả về 33.

2.2 Số Fibonacci

Làm việc với dãy Fibonacci chuẩn $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$. Input gồm N truy vấn và một modulo cố định Q ($1 \leq N \leq 10^6$, $1 \leq Q \leq 10^9$, $0 \leq A_i \leq 2 \times 10^8$). Với mỗi chỉ số A_i , tính $F(A_i) \bmod Q$. Hàm **MAIN** phải trả về danh sách độ dài N chứa kết quả của từng truy vấn theo đúng thứ tự.

Input: Ví dụ

```
3 1000000000
3
4
5
```

trong đó dòng đầu chứa N và Q , các dòng tiếp theo là A_i .

Output: Danh sách $[2, 3, 5]$ tương ứng $F(3)$, $F(4)$, $F(5)$ modulo 10^9 .

2.3 Nhân hai ma trận

Cho hai ma trận vuông A và B kích thước $N \times N$ ($1 \leq N \leq 10^4$, phần tử trong $[-10^9, 10^9]$). Cần tính $C = A \times B$ và đồng thời trích xuất: (1) đường chéo chính của C , (2) đường chéo phụ của C , (3) tổng toàn bộ phần tử của C . Hàm **MAIN** trả về một cấu trúc chứa ma trận C , hai danh sách đường chéo và giá trị tổng, giúp nhóm dễ kiểm chứng độ đúng đắn trước khi phân tích song song.

Input:

```
2
1 2
3 4
5 6
7 8
```

trong đó ma trận A chiếm N dòng tiếp theo sau N , kế đến là ma trận B .

Output: Trả về cấu trúc chứa $C = [[19, 22], [43, 50]]$, **diag_main** = $[19, 50]$, **diag_secondary** = $[22, 43]$, **total_sum** = 134.

2.4 Sắp xếp mảng một chiều

Có N bản ghi dạng $(v, label)$ với $v \in [-10^9, 10^9]$ và **label** là chuỗi không có khoảng trắng ($1 \leq N \leq 10^6$). Yêu cầu sắp xếp ổn định theo các tiêu chí: v giảm dần, nếu trùng v thì so sánh **label** tăng dần từ điển, nếu cả hai cùng trùng phải giữ nguyên thứ tự ban đầu. Không được dùng các hàm sắp xếp dựng sẵn của Python; thay vào đó cần tự hiện thực thuật toán (ưu tiên song song) và trả về danh sách đã sắp xếp.

Input:

```
4
```

5 QA
 5 QB
 -1 ID
 9 QA

Output: Danh sách $[(9, QA), (5, QA), (5, QB), (-1, ID)]$.

2.5 Tìm kiếm trong mảng số nguyên lớn

Input là khóa cần tìm K và một dãy số nguyên rất lớn (tối đa 10^6 phần tử) phân tách bởi khoảng trắng hoặc xuống dòng. Đọc toàn bộ vào mảng A , triển khai chiến lược tìm kiếm song song để xác định chỉ số nhỏ nhất i sao cho $A[i] = K$. Nếu không tồn tại trả về -1 . Không được sử dụng phép `in` hay các hàm sắp xếp/tìm kiếm dựng sẵn; các nhóm phải tự chia mảng thành nhiều đoạn và phối hợp giữa các worker để đảm bảo thời gian dưới 60 giây.

Input:

20
 10 25 5 30 15 40 20 5 35 45 0

trong đó dòng đầu là K , các dòng sau là mảng.

Output: Chỉ số đầu tiên chứa K (ví dụ trên trả về 6 nếu đánh số từ 0) hoặc -1 nếu không tìm thấy.

2.6 Phân tích không gian nhúng

Có Q bộ dữ liệu ($1 \leq Q \leq 100$), mỗi bộ gồm N điểm 2D ($1 \leq N \leq 3 \times 10^5$, $|x_i|, |y_i| \leq 10^9$) biểu diễn các từ sau khi giảm chiều từ mô hình nhúng. Với từng bộ, hãy tìm khoảng cách Euclid nhỏ nhất giữa hai điểm bất kỳ (cặp từ tương đồng nhất) bằng một thuật toán hiệu quả, có thể chia để trị kết hợp song song. Hàm `MAIN` trả về danh sách Q số thực đã làm tròn 4 chữ số thập phân, trong đó phần tử thứ i tương ứng với bộ dữ liệu thứ i .

Input:

2
 4
 4 0
 8 9
 0 4
 2 0
 4
 2 0
 1 6
 3 9
 3 10

Output: Danh sách $[2.0, 1.0]$, nghĩa là khoảng cách nhỏ nhất ở bộ đầu là 2 đơn vị, bộ thứ hai là 1 đơn vị.

3 Yêu cầu trong báo cáo cho challenge 1, 2, 3, 4, 5, 6

Sinh viên thực hiện các mục sau cho từng challenge:

- **Phân tích song song hóa:**

- Xác định các phần công việc có thể thực hiện song song trong thuật toán ở các thử thách
- Đề xuất một hoặc nhiều phương pháp phân chia công việc cho các tiến trình/luồng xử lý cho từng bài toán
- Mô tả chi tiết cách dữ liệu được phân phối và truy cập bởi các tiến trình/luồng
- Phân tích các yếu tố có thể ảnh hưởng đến hiệu năng của chương trình song song, chẳng hạn như sự phụ thuộc dữ liệu, cân bằng tải, và chi phí giao tiếp (nếu có).

- **Thiết kế thuật toán song song:**

- Trình bày thuật toán song song chi tiết cho từng challenge
- Sử dụng sơ đồ hoặc lưu đồ (nếu cần) để minh họa luồng thực hiện của thuật toán

- **Đánh giá hiệu năng:**

- Thực hiện đo thời gian thực thi của chương trình song song. Với bài toán nhân 2 ma trận thử các kích thước ma trận khác nhau (ví dụ $n = 128, 256, 512, 1024, 5000$). Với bài toán sắp xếp thử các kích thước mảng khác nhau (ví dụ $n = 10^4, 10^5, 10^6$)
- So sánh hiệu năng của chương trình song song với một chương trình tuần tự tương ứng
- Giải thích các kết quả thu được và thảo luận về các yếu tố ảnh hưởng đến hiệu năng

4 Nộp bài và đánh giá

- **Mã nguồn**

- Các nhóm nộp mã nguồn Python cho bài toán lập trình song song đã viết (bản cuối cùng) lên hệ thống đã cung cấp, **KHÔNG NỘP MÃ NGUỒN CHẠY TUẦN TỰ LÊN HỆ THỐNG**. Sử dụng key đã cấp cho từng nhóm để nộp bài. Mã nguồn cần qua hết các tests. Mã nguồn cần nộp lên hệ thống trước ngày báo cáo
- **Mã nguồn cần chạy trên server:** Pass all test
- Nộp mã nguồn theo các challenges tương ứng với ID: https://challenges.iec-uit.com/challenge_
- Các bạn xem leaderboard của challenges tương ứng với ID: https://challenges.iec-uit.com/challenge_<id>/leaderboard

- **Báo cáo và trình bày**

- **Báo cáo:** Trả lời các yêu cầu như đã hướng dẫn. Ghi rõ mã nhóm + key + thông tin các thành viên. Nộp báo cáo theo định dạng **PDF**. Báo cáo sẽ được nộp lên website môn học cùng deadline với nộp mã nguồn.
- **Nội dung trình bày:** Sinh viên tự chuẩn bị slide để trình bày ở buổi báo cáo. Nội dung trình bày gồm:
 - * Phương pháp phân chia công việc cho các tiến trình/luồng xử lý cho các bài toán trên
 - * Mô tả chi tiết cách dữ liệu được phân phối và truy cập bởi các tiến trình/luồng theo phương pháp đã chọn
 - * Trình bày thuật toán song song một cách rõ ràng và chi tiết, dựa trên phương pháp phân chia công việc đã chọn
 - * Sử dụng sơ đồ hoặc lưu đồ (nếu cần thiết) để minh họa các bước thực hiện của thuật toán và sự tương tác giữa các tiến trình/luồng
 - * Giải thích vai trò của từng bước trong thuật toán và cách chúng đóng góp và việc tính toán ma trận kết quả và sắp xếp mảng
 - * Tiềm năng tăng tốc (speedup) có thể đạt được khi sử dụng thuật toán song song với thuật toán tuần tự
 - * Ví dụ minh họa chi tiết
- **Đánh giá mỗi challenge:**
 - * **[8 điểm]** Tính điểm từng challenge dựa trên thời gian thực thi của mã nguồn hiện thực thuật toán song song trên hệ thống:
 - 3 nhóm đứng đầu được tối đa điểm của bài (8 điểm)
 - 5 nhóm tiếp theo trừ 1 điểm (7 điểm)
 - 5 nhóm tiếp theo trừ 2 điểm (6 điểm)
 - 5 nhóm tiếp theo trừ 3 điểm (5 điểm)
 - * **[2 điểm]** Nộp báo cáo report + chuẩn bị slide thuyết trình (về các giải thuật song song của từng challenge) ở buổi thực hành tiếp theo.

5 Danh sách thư viện được phép dùng

Hệ thống đã **import sẵn** bộ thư viện dưới đây. Sinh viên chỉ việc sử dụng trực tiếp, tuyệt đối **không tự import lại** vì hệ thống chấm không cho phép!!!

- **Song song và lập lịch:** `time`, `threading`, `multiprocessing`, `concurrent.futures`, `asyncio`, `joblib`.
- **Xử lý chuỗi/dữ liệu:** `string`, `re`, `collections`, `queue`.
- **Toán học và ngẫu nhiên:** `math`, `cmath`, `decimal`, `fractions`, `random`.

Lưu ý: Bài sao chép, nộp trễ, “gánh team”, ... sẽ được xử lý tùy mức độ.